



TP 8 – Rendre le catalogue interactif localement

State Management : Les états Compose

1. Objectif du TP

Dans le TP précédent, vous avez personnalisé graphiquement **Product Explorer** avec Material Design 3.

L'application affiche déjà un catalogue local de produits.

Dans ce TP, vous allez rendre cet écran interactif avec les premiers outils d'état Compose :

- `remember` ;
- `mutableStateOf` ;
- `rememberSaveable`.

À la fin du TP, l'utilisateur pourra :

- **saisir** un texte de recherche ;
- **filtrer** les produits affichés ;
- **afficher uniquement** les produits en stock ;
- **marquer** des produits comme **favoris** ;
- voir l'interface se mettre à jour **automatiquement**.

Les données restent locales.

2. Point de départ

Vous repartez du code du TP7.

Vous avez normalement un écran proche de celui-ci (dans `MainActivity`) :

```
ProductCatalogScreen(  
    products = sampleProducts(),  
    onProductClick = {  
        // Plus tard : ouvrir le détail du produit  
    },  
    modifier = Modifier.padding(innerPadding)  
)
```

Dans ce TP, nous allons modifier principalement :

- `ProductCatalogScreen` ;
- `ProductListItem`.

3. Étape 1 – Ajouter un état pour la recherche

Une **recherche** a besoin de mémoriser ce que l'utilisateur saisit et cette information **peut changer pendant** l'utilisation de l'application. *C'est donc un **état**.*

Nous allons commencer avec `rememberSaveable`, car le texte de recherche est une **donnée simple** que l'on veut conserver si l'écran est recréé.

Dans `ProductCatalogScreen`, ajoutez au début de la fonction (dans le corps) :

```
var searchQuery by rememberSaveable {  
    mutableStateOf("")  
}
```



Imports nécessaires

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
```

`searchQuery` c'est une variable qui va garder en mémoire le texte saisi par l'utilisateur dans la barre de recherche. Quand cette valeur change, Compose peut mettre à jour l'interface.

- `mutableStateOf("")` crée une valeur modifiable, vide au départ.
- `rememberSaveable` garde cette valeur même si l'écran est recréé, par exemple après une rotation.
- Quand `searchQuery` change, Compose détecte le changement et peut redessiner l'interface avec la nouvelle valeur.

4. Étape 2 – Ajouter un champ de recherche

Nous allons afficher un `OutlinedTextField` en haut du catalogue.

Cette fois, le champ n'est plus en lecture seule - sa valeur dépend de `searchQuery`.

Quand l'utilisateur saisit du texte, `onValueChange` met à jour l'état.

Alors, dans la `LazyColumn`, après le titre, ajoutez :

```
item {
    OutlinedTextField(
        value = searchQuery,
        onValueChange = { newValue ->
            searchQuery = newValue
        },
        label = {
            Text(text = "Rechercher un produit")
        },
        modifier = Modifier.fillMaxWidth()
    )
}
```

Import si nécessaire

```
import androidx.compose.material3.OutlinedTextField
```

Le champ affiche toujours la valeur de `searchQuery`.

Quand l'utilisateur tape, `searchQuery` change.

L'interface peut alors être **recomposée**.

5. Étape 3 – Filtrer les produits avec *remember*

Nous voulons afficher seulement les produits dont le **titre**, la **marque** ou la **catégorie** correspond au texte saisi.

Le résultat filtré dépend de deux choses :

- la liste complète `products` ;
- la recherche `searchQuery`.

Nous allons calculer cette liste avec `remember`.

Toujours dans `ProductCatalogScreen`, sous l'état `searchQuery`, ajoutez :



```
val filteredProducts = remember(products, searchQuery) {  
    products.filter { product ->  
        product.title.contains(searchQuery, ignoreCase = true) ||  
        product.brand.contains(searchQuery, ignoreCase = true) ||  
        product.category.contains(searchQuery, ignoreCase = true)  
    }  
}
```

Import nécessaire

```
import androidx.compose.runtime.remember
```

remember évite de recalculer inutilement la liste à chaque **recomposition**.

La liste est recalculée seulement si **products** ou **searchQuery** change.

6. Étape 4 – Utiliser la liste filtrée

Pour l’instant, votre **LazyColumn** affiche probablement encore **products**, à la place nous devons afficher **filteredProducts**.

Pour cela remplacez :

```
items(  
    items = products,  
    key = { product -> product.id }  
) { product ->
```

par :

```
items(  
    items = filteredProducts,  
    key = { product -> product.id }  
) { product ->
```

La liste affichée dépend maintenant de l’état de **recherche**.

Si la recherche est vide, tous les produits sont affichés.

Si l’utilisateur tape un mot, la liste se réduit.

7. Étape 5 – Ajouter un état « produits en stock uniquement »

Nous allons ajouter un **deuxième filtre**.

L’utilisateur pourra **choisir** d’afficher seulement les produits **disponibles**.

Cette information est un **booléen** : *vrai* ou *faux*.

Commencez par ajoutez cet état sous **searchQuery** :

```
var showOnlyInStock by rememberSaveable {  
    mutableStateOf(false)  
}
```

Puis **modifiez** le calcul de **filteredProducts** pour y ajouter ce troisième paramètre :



```
val filteredProducts = remember(products, searchQuery, showOnlyInStock) {
    products.filter { product ->
        val matchesSearch =
            product.title.contains(searchQuery, ignoreCase = true) ||
            product.brand.contains(searchQuery, ignoreCase = true) ||
            product.category.contains(searchQuery, ignoreCase = true)

        val matchesStock =
            !showOnlyInStock || product.stock > 0

        matchesSearch && matchesStock
    }
}
```

Dans ce code :

- Si `showOnlyInStock` vaut *false*, on affiche **tous les produits**.
- Si `showOnlyInStock` vaut *true*, on garde seulement les produits dont le stock est supérieur à zéro.

8. Étape 6 – Ajouter un bouton de filtre

Nous allons maintenant permettre à l'utilisateur d'**activer** ou **désactiver** ce filtre.

Pour rester simple, nous utilisons un `Button`.

Ajoutez cet élément dans la `LazyColumn`, après le champ de recherche :

```
item {
    Button(
        onClick = {
            showOnlyInStock = !showOnlyInStock
        }
    ) {
        Text(
            text = if (showOnlyInStock) {
                "Afficher tous les produits"
            } else {
                "Afficher uniquement les produits en stock"
            }
        )
    }
}
```

Le bouton inverse la valeur de `showOnlyInStock`.

Quand cette valeur change, `filteredProducts` est recalculée.

L'écran affiche donc **automatiquement** la nouvelle liste.

9. Étape 7 – Afficher le nombre de résultats

Il est utile d'indiquer à l'utilisateur **combien de produits** sont affichés.

Cette information dépend directement de `filteredProducts`.

Pour l'afficher, ajoutez cet élément avant la liste des produits :

```
item {
    Text(
        text = "${filteredProducts.size} produit(s) affiché(s)",
        style = MaterialTheme.typography.bodyMedium
    )
}
```

Ce texte change automatiquement quand la recherche ou le filtre change.

Il n'y a pas besoin de le modifier manuellement.



Il dépend simplement de l'état courant.

10. Étape 8 – Ajouter un état local pour les favoris

Nous allons maintenant permettre à l'utilisateur de **marquer des produits comme favoris**.

Pour rester simple, nous allons mémoriser les identifiants des produits **favoris**.

Nous utilisons une liste d'identifiants.

Ajoutez cet état dans **ProductCatalogScreen** :

```
var favoriteProductIds by rememberSaveable {  
    mutableStateOf(ListOf<Int>())  
}
```

favoriteProductIds contient les **id** des produits favoris. Par exemple : **[1, 3]**, signifie que les produits **1** et **3** sont favoris.

11. Étape 9 – Créer une fonction de modification des favoris

Pour éviter de répéter le code, nous allons créer une petite **fonction locale** qui ajoutera ou retirera un produit des favoris.

Sous les états, ajoutez :

```
fun toggleFavorite(productId: Int) {  
    favoriteProductIds = if (favoriteProductIds.contains(productId)) {  
        favoriteProductIds - productId  
    } else {  
        favoriteProductIds + productId  
    }  
}
```

Si le produit est déjà favori, on le retire ; sinon, on l'ajoute.

Important : on crée une nouvelle liste à chaque modification.

Cela permet à Compose de détecter le changement.

12. Étape 10 – Modifier la carte produit

La carte produit doit maintenant savoir si le produit est **favori** et proposer un bouton pour modifier cet état.

Nous allons donc ajouter deux paramètres à **ProductListItem** :

- **isFavorite** ;
- **onFavoriteClick**.

Modifiez la signature de **ProductListItem** pour ajouter ces 2 paramètres ainsi :

```
@Composable  
fun ProductListItem(  
    product: ProductUi,  
    isFavorite: Boolean,  
    onFavoriteClick: () -> Unit,  
    onClick: () -> Unit,  
    modifier: Modifier = Modifier  
) {
```

Puis, dans le contenu de la carte, ajoutez ce bouton avant ou après « **Voir le produit** » :



```
Button(onClick = onFavoriteClick) {  
    Text(  
        text = if (isFavorite) {  
            "Retirer des favoris"  
        } else {  
            "Ajouter aux favoris"  
        }  
    )  
}
```

La carte ne garde pas elle-même la liste des favoris.

Elle reçoit simplement : *l'information à afficher* et *l'action à déclencher*.

Pour cette séance, l'état reste dans l'écran catalogue.

13. Étape 11 – Brancher les favoris dans la liste

Dans `ProductCatalogScreen`, chaque carte doit recevoir son état favori.

Elle doit aussi appeler `toggleFavorite`.

Dans le bloc `items`, modifiez l'appel à `ProductListItem` :

```
ProductListItem(  
    product = product,  
    isFavorite = favoriteProductIds.contains(product.id),  
    onFavoriteClick = {  
        toggleFavorite(product.id)  
    },  
    onClick = {  
        onProductClick(product)  
    }  
)
```

Chaque produit vérifie si son `id` est présent dans la liste des favoris.

Quand on clique sur le bouton favori, *l'état change*.

Compose met ensuite à jour l'affichage.

14. Étape 12 – Version finale de avec `ProductCatalogScreen` et `ProductListItem` modifiés

Voici une version finale possible de l'écran.

Elle reste volontairement locale : *l'état est directement dans le composable*.

Dans le TP suivante, nous verrons comment améliorer cette organisation.

Voici la version finale du code (`MainActivity.kt`) :

<https://gist.github.com/ariatsi/de87a27f20c102ae44f1dabeff1d8a0e> .

Adaptez comme d'habitude le nom du package et le nom du thème à votre projet.

Et voici le répertoire GitHub avec cette version :

<https://github.com/ariatsi/ProductExplorer/releases/tag/tp8-final> .

15. À tester

Testez les cas suivants :

1. Saisir "audio" dans la recherche.
2. Saisir le nom d'une marque.
3. Effacer la recherche.



4. Activer le filtre "en stock uniquement".
5. Désactiver le filtre.
6. Ajouter un produit aux favoris.
7. Retirer un produit des favoris.
8. Tourner l'écran si vous testez sur émulateur ou téléphone.

Observez que les valeurs gérées avec `rememberSaveable` peuvent être conservées lors d'une recréation simple de l'écran.

16. Ce qu'il faut retenir

Dans ce TP, vous avez rendu le catalogue **Product Explorer** interactif localement.

Vous avez utilisé :

- `mutableStateOf` pour créer un état observable ;
- `remember` pour mémoriser un calcul dépendant de certaines valeurs ;
- `rememberSaveable` pour conserver des états simples ;
- un champ de **recherche** interactif ;
- un **filtre** local ;
- une liste de **favoris** locale.

Le point essentiel est le suivant :

En Compose, on ne modifie pas directement l'interface.

On modifie l'état, et l'interface se met à jour à partir de cet état.

Dans la séance suivante, nous améliorerons l'organisation de ce code avec le **State Hoisting**.